

```

1: //Linear Queue using Array
2: #include <stdio.h>
3: #define MAX_SIZE 100
4: int queue[MAX_SIZE];
5: //Initially when the queue is empty, front and rear will be -1
6: //On every insertion, rear will be increased
7: //On every deletion, front will be increased
8: int front = -1;
9: int rear = -1;
10:
11: //Insertion in queue is known as enqueue
12: void enqueue(int value)
13: {
14:     if (rear == MAX_SIZE - 1)
15:     {
16:         printf("Queue is full. Cannot enqueue element.\n");
17:         return;
18:     }
19:     if (front == -1)
20:         front = 0;
21:     rear++;
22:     queue[rear] = value;
23: }
24: /*
25: //Deletion in queue is known as dequeue
26: //Queue will be empty in two situations:
27: //1. When front will be -1
28: //2. When front>rear. This will occur when we delete the only left
   element in the queue
29: //When there is only one element in the queue, front and rear will
   point to the same element.
30: //If we delete that element also, front will be incremented.So,
   (front>rear i.e front=rear+1)
31: //and in that situation, list will be empty.
32: */
33: void dequeue()
34: {
35:     if (front == -1 || front > rear)
36:     {
37:         printf("Queue is empty. Cannot dequeue element.\n");
38:         return;
39:     }
40:     printf("Dequeued element: %d\n", queue[front]);
41:     front++;
42: }
43:
44: void display()
45: {
46:     if (front == -1 || front > rear)

```

```

47:     {
48:         printf("Queue is empty.\n");
49:         return;
50:     }
51:     printf("Queue elements: ");
52:     for (int i = front; i <= rear; i++)
53:     {
54:         printf("%d ", queue[i]);
55:     }
56:     printf("\n");
57: }
58:
59: int main()
60: {
61:     enqueue(10);
62:     enqueue(20);
63:     enqueue(30);
64:     enqueue(40);
65:
66:     display();
67:
68:     dequeue();
69:     dequeue();
70:     dequeue();
71:     dequeue();
72:
73:     display();
74:
75:     return 0;
76: }
77:
78:
79: //Linear Queue using Linked List
80: #include <stdio.h>
81: #include <stdlib.h>
82: // Structure to create a node with data and next pointer
83: struct node
84: {
85:     int data;
86:     struct node *next;
87: };
88:
89: struct node *front = NULL;
90: struct node *rear = NULL;
91:
92: // Enqueue() operation on a queue, Insertion in a queue at the rear
93: void enqueue(int value)
94: {
95:     struct node *ptr;

```

```

96:     ptr = (struct node *)malloc(sizeof(struct node));
97:     ptr->data = value;
98:     ptr->next = NULL;
99:     if ((front == NULL) && (rear == NULL))
100:    {
101:        front = rear = ptr;
102:    }
103:     else
104:    {
105:        rear->next = ptr;
106:        rear = ptr;
107:    }
108:     printf("Node is Inserted\n\n");
109: }
110:
111: // Dequeue() operation on a queue, Deletion in queue at front
112: int dequeue()
113: {
114:     if (front == NULL)
115:     {
116:         printf("\nUnderflow\n");
117:         return -1;
118:     }
119:     else
120:     {
121:         struct node *temp = front;
122:         int temp_data = front->data;
123:         front = front->next;
124:         free(temp);
125:         return temp_data;
126:     }
127: }
128:
129: // Display all elements of queue
130: void display()
131: {
132:     struct node *temp;
133:     if ((front == NULL) && (rear == NULL))
134:     {
135:         printf("\nQueue is Empty\n");
136:     }
137:     else
138:     {
139:         printf("The queue is \n");
140:         temp = front;
141:         while (temp)
142:         {
143:             printf("%d--->", temp->data);
144:             temp = temp->next;

```

```

145:     }
146:     printf("NULL\n\n");
147: }
148: }
149:
150: int main()
151: {
152:     int choice, value;
153:     printf("\nImplementation of Queue using Linked List\n");
154:     while (choice != 4)
155:     {
156:         printf("1.Enqueue\n2.Dequeue\n3.Display\n4.Exit\n");
157:         printf("\nEnter your choice : ");
158:         scanf("%d", &choice);
159:         switch (choice)
160:         {
161:             case 1:
162:                 printf("\nEnter the value to insert: ");
163:                 scanf("%d", &value);
164:                 enqueue(value);
165:                 break;
166:             case 2:
167:                 printf("Popped element is :%d\n", dequeue());
168:                 break;
169:             case 3:
170:                 display();
171:                 break;
172:             case 4:
173:                 exit(0);
174:                 break;
175:             default:
176:                 printf("\nWrong Choice\n");
177:         }
178:     }
179:     return 0;
180: }

```